

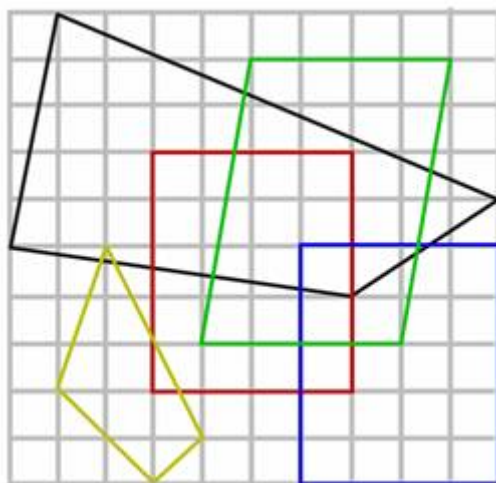
UVa11139 Counting Quadrilaterals

题目链接

[UVa11139 Counting Quadrilaterals](#)

题意

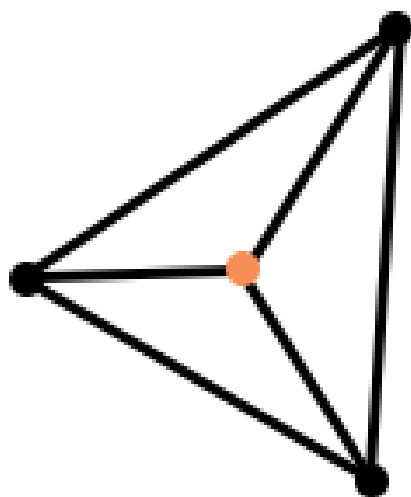
在图下图的 10×10 网格中，你可以看到 5 个四边形（注意，四边形的 4 条边不能相交，而且没有三点共线）。



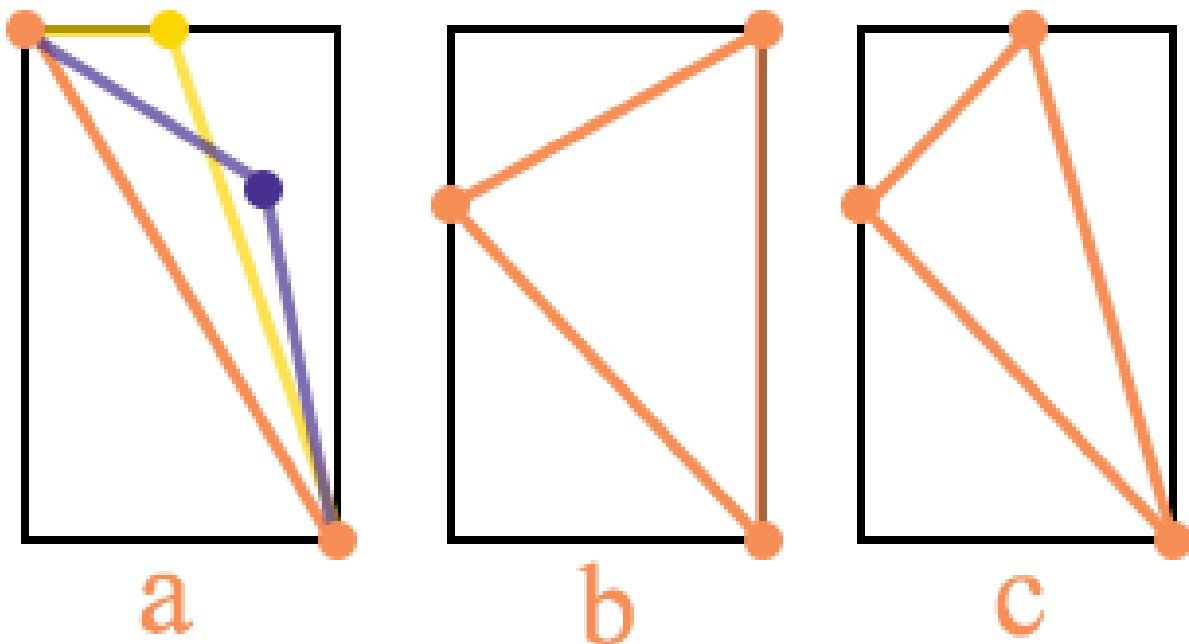
当然，还可以连出其他网格四边形。编程统计出 $n \times n$ ($n \leq 120$) 网格里可以连出多少个网格四边形。如 $n=2$ 时有 94 个， $n=10$ 时有 12046294 个。

分析

四边形可按凹凸分类，先考虑凹四边形，它必然是一个三角形和其内一点形成的（见下图），并且三角形内任意一点与三角形的三个顶点能组成三个凹四边形，因此统计凹四边形可以枚举格点三角形和其内格点数（具体需要用到[皮克定理 Pick's theorem](#)）。



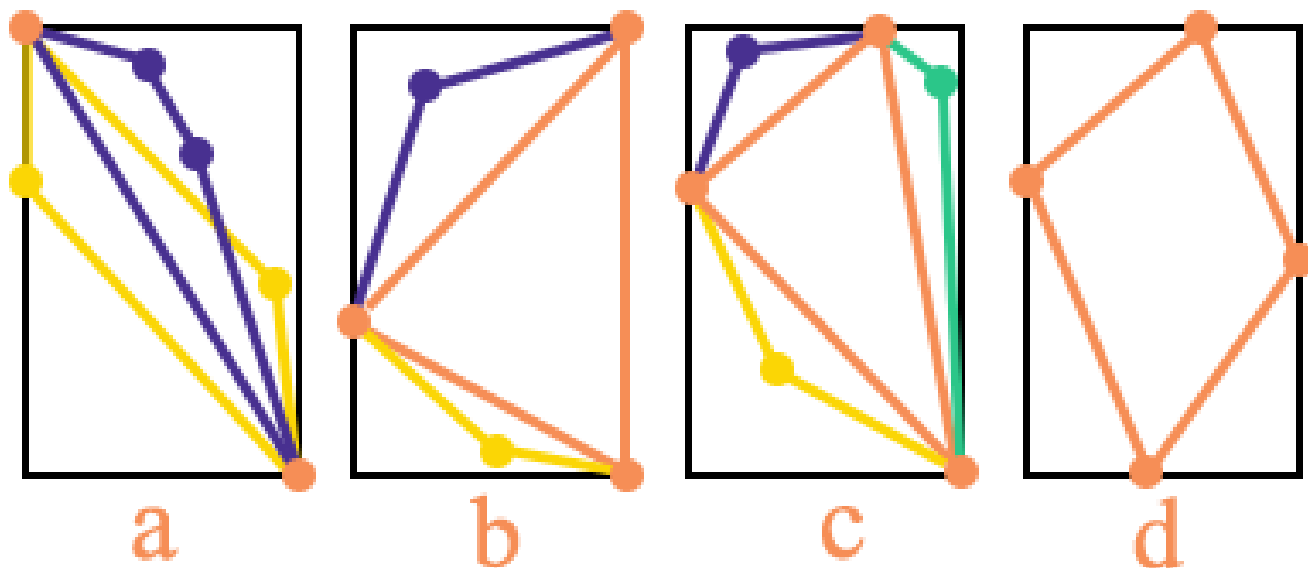
枚举格点三角形又可以按照三角形和其外包矩形的关系来进行：



- 三角形有两个顶点是外包矩形对角线的端点，第三个顶点为不在此对角线上的任意一点；
- 三角形有两个顶点是外包矩形某条边的端点，第三个顶点为此边的平行边上不在端点的任意一点；
- 三角形一个顶点是外包矩形的角顶点，另外两个顶点在与此角不相连的两边上任意一点且非端点；

设 $c[m][n]$ 表示以尺寸为 $m \times n$ 为外包矩形的凹四边形计数（后面还会加上凸四边形计数），则对给定的 m 、 n 可以按照上面三种情况累加计数。

接下来考虑凸四边形和其外包矩形的关系：

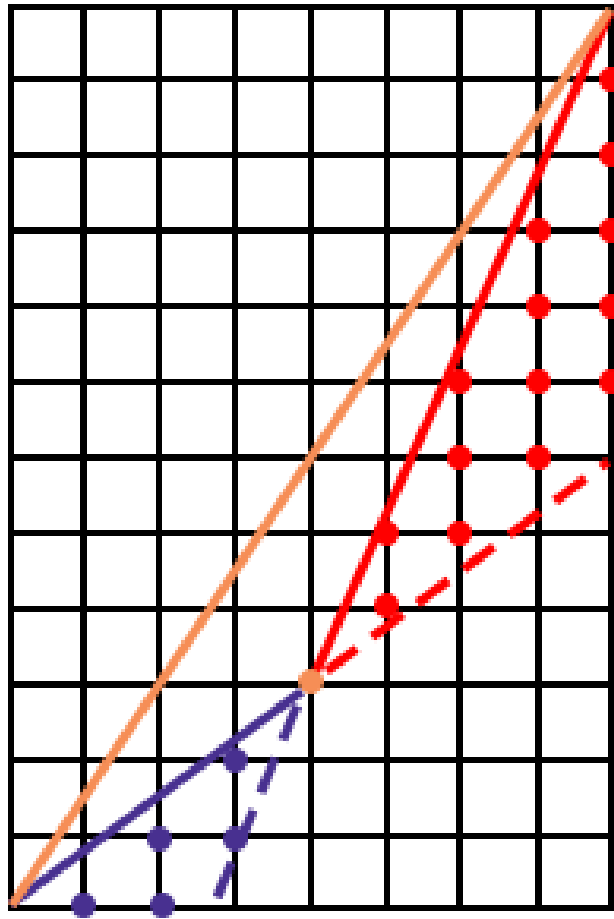


和凹四边形的分类相似，但多出一种情况 d：四个顶点各在一条边上。

凸四边形的 d 类计数的结果为 $(m - 1)^2 \times (n - 1)^2$;

b、c 分类的计数复杂一点，首先需要把第四个顶点在外包矩形边上时单独考虑，在外包矩形内部时借助[皮克定理](#)可求；

a 分类的计算是最复杂的，对于另外两个端点跨对角线两边（图 a 所示黄色的四边形）好计算，但是在对角线同侧（图 a 所示紫色的四边形）时则复杂，下面详细展开：



可以先枚举其中一个点，它与两对角点连线并延伸见上图所示，可见只有连线 and 延伸线所夹的格点才能与之构成凸四边形，可惜这些点的计数不能使用[皮克定理](#)求解了，因为延伸线与外包矩形的边界交点坐标可能不是整数，不构成格点多边形这一前提条件。

这种可以借助动态规划单独计算出来：设 $f[w][h][s]$ 表示宽高比为 $w:h$ (w, h 互质) 并且横向宽度为 s 的斜线下方的格点计数，那么 $f[w][h][s+1] = f[w][h][s] + \sigma(s)$ ，这里当 $(s+1) \times h \% w == 0$ 时 $\sigma(s) = (s+1) \times h / w - 1$ 否则 $\sigma(s) = \lfloor (s+1) \times h / w \rfloor$ 。

实际的计算过程及一些细节请参见 ac 代码。

AC 代码

```
1  #include <iostream>
2  using namespace std;
3
4  #define N 123
5  int f[N][N][N], g[N][N], n; long long c[N][N];
6
```

C/C++

```

7  int gcd(int a, int b) {
8      if (a > b) return gcd(b, a);
9      if (a == 0) return b;
10     if (a & 1) {
11         if (b & 1) return gcd(a, (b-a)>>1);
12         return gcd(a, b >> 1);
13     } else if (b & 1) return gcd(a >> 1, b);
14     return gcd(a >> 1, b >> 1) << 1;
15 }
16
17 void init() {
18     for (int i=0; i<N; ++i) for (int j=i; j<N; ++j) g[i][j] = g[j][i] = gcd(i, j);
19     for (int i=0; i<N; ++i) for (int j=0; j<N; ++j) {
20         f[i][j][0] = 0;
21         for (int k=1; k<N; ++k) f[i][j][k] = f[i][j][k-1] + (i ? (k*j%i ? k*j/i :
22             k*j/i-1) : 0);
23     }
24     for (int x=1; x<N; ++x) for (int y=x; y<N; ++y) {
25         long long t = (x+1)*(y+1) - g[x][y] - 1;
26         long long cc = t*t/2-1 + (x-1)*(x-2) + (y-1)*(y-2) + (x-1)*(y-1)*(6*(x+
27             y) - 8) + (x-1)*(x-1)*(y-1)*(y-1);
28         for (int x1=1; x1<=x; ++x1) {
29             int y1 = y*x1; y1 = y1%x ? y1/x : y1/x-1;
30             while (y1 >= 0) {
31                 cc += 2*(f[y1][x1][y1] - f[y-y1][x-x1][y1] - g[y-y1][x-x1]*y1/(y-y
32                     1));
33                 cc += 2*(f[x-x1][y-y1][x-x1] - f[x1][y1][x-x1] - g[x1][y1]*(x-x1)/x
34                     1);
35                 cc += 6*(x1*y-x*y1 + 2 - g[x1][y1] - g[x-x1][y-y1] - g[x][y]);
36                 --y1;
37             }
38         }
39         for (int x1=1; x1<x; ++x1) {
40             cc += (x-2)*(y-1) + 2 - g[x1][y] - g[x-x1][y];
41             cc += 3*(x*y + 2 - x - g[x1][y] - g[x-x1][y]);
42         }
43         for (int y1=1; y1<y; ++y1) {
44             cc += (x-1)*(y-2) + 2 - g[x][y1] - g[x][y-y1];
45             cc += 3*(x*y + 2 - y - g[x][y1] - g[x][y-y1]);
46         }
47         for (int x1=1; x1<x; ++x1) for (int y1=1; y1<y; ++y1) {
48             cc += 2*(x*y+x1*y1 + 6 - 2*x - 2*y - g[x1][y] - g[x][y1] - g[x-x1][y-y
49                 1]);
50             cc += 6*(x*y-x1*y1 + 2 - g[x1][y] - g[x][y1] - g[x-x1][y-y1]);
51         }
52         c[x][y] = c[y][x] = cc;
53     }
54 }

```

```
50
51 void solve() {
52     long long ans = 0;
53     for (int x=1; x<=n; ++x) for (int y=1; y<=n; ++y) ans += (n+1-x)*(n+1-y) * c
[x][y];
54     cout << n << ' ' << ans << endl;
55 }
56
57 int main() {
58     init();
59     while (cin>>n && n) solve();
60     return 0;
61 }
```